

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami :

1. Mahasiswa mampu menguasai teori fundamental operasional *Transaction Control Language* (TCL) dalam mengelola unit kerja transaksi *database*.
2. Mahasiswa mampu menggunakan fungsi `COMMIT` untuk mengunci dan meresmikan perubahan baris data ke dalam kamus data basis data.
3. Mahasiswa mampu menggunakan fungsi `ROLLBACK` untuk memulihkan status data kembali ke kondisi awal sebelum transaksi aktif dimulai.
4. Mahasiswa mampu membuat titik simpan logis di dalam sesi transaksi aktif menggunakan instruksi `SAVEPOINT`.
5. Mahasiswa mampu mempraktikkan pembatalan transaksi bersyarat menggunakan perintah `ROLLBACK TO SAVEPOINT` menuju titik simpan yang telah ditentukan.
6. Mahasiswa mampu mengeksekusi perintah manipulasi data secara aman di bawah pengawasan kontrol transaksional.
7. Mahasiswa mampu memahami mekanisme kerja memori sementara (buffer) *database* yang menahan perubahan data agar tidak langsung tersimpan sebelum verifikasi akhir.

12.2 Dasar Teori

Transaction Control Language (TCL) merupakan sekelompok instruksi khusus dalam SQL yang difungsikan untuk mengatur jalannya unit kerja transaksional di dalam sistem manajemen basis data. Sebuah transaksi sendiri direpresentasikan sebagai satu blok logis yang membundel satu atau sekumpulan perintah manipulasi data (DML) agar dapat dieksekusi sebagai satu kesatuan kerja yang utuh. Melalui implementasi kontrol transaksi ini, sistem dapat menjamin bahwa seluruh rangkaian modifikasi informasi yang terjadi di dalam memori tidak akan langsung diterapkan ke dalam penyimpanan fisik secara permanen sebelum divalidasi oleh pengguna (Mishra, 2021).

Instrumen utama yang digunakan untuk meresmikan setiap pembaruan struktur nilai di dalam basis data adalah perintah `COMMIT`. Ketika instruksi ini dieksekusi, seluruh perubahan data yang sebelumnya berstatus tertunda (*pending*) di dalam memori transaksional akan langsung dikunci secara permanen ke dalam sistem penyimpanan fisik basis data, sekaligus menandai akhir dari siklus hidup transaksi yang aktif. Dampak penting dari penerapan komitmen data ini adalah terbukanya hak visibilitas informasi, di mana data baru yang telah disahkan tersebut kini dapat diakses dan dilihat secara langsung oleh seluruh pengguna lain yang terhubung ke dalam jaringan server (Forta, 2024).

Sebagai langkah mitigasi terhadap potensi kesalahan operasional selama proses manipulasi kueri berlangsung, pengguna dapat memanfaatkan perintah `ROLLBACK` untuk memulihkan kondisi data. Jika instruksi ini dijalankan secara mandiri, database akan membatalkan seluruh urutan perubahan kueri DML yang belum dikomit dan mengembalikan status data ke kondisi awal sebelum transaksi aktif tersebut dimulai. Namun, jalannya transaksi dapat diorganisasi secara lebih fleksibel menjadi beberapa tahapan kecil melalui penandaan memori sementara menggunakan perintah `SAVEPOINT`. Keberadaan titik simpan sementara ini memungkinkan pengembang mengeksekusi perintah `ROLLBACK TO SAVEPOINT` untuk memulihkan data menuju titik penanda tertentu secara parsial, tanpa harus mengorbankan atau mengulang kembali seluruh rangkaian kueri yang sudah berjalan dengan benar (Connolly & Begg, 2022).

Dalam lingkungan basis data yang multi-pengguna (*multi-user*), stabilitas pembacaan informasi dikawal ketat oleh mekanisme *read consistency* guna memastikan setiap pengguna menerima representasi data yang stabil dan konsisten. Ketika seorang pengguna sedang melakukan modifikasi terhadap suatu baris data, sistem secara cerdas akan mengisolasi perubahan tersebut sehingga pengguna lain tetap melihat data asli dalam kondisi sebelum perubahan disetujui. Sinkronisasi ini diperkuat oleh fitur *locks* atau penguncian data otomatis yang diterapkan oleh mesin database untuk memblokir upaya manipulasi ganda terhadap baris data atau objek yang sama pada waktu bersamaan. Proteksi penguncian ini akan terus dipertahankan demi menghindari konflik antartransaksi, dan baru akan dilepaskan secara otomatis setelah transaksi tersebut diselesaikan (Silberschatz et al., 2024).

12.3 Hasil dan Pembahasan

Berikut adalah hasil dan pembahasan dari percobaan *query* yang sudah dilakukan dalam kelas praktikum:

1. *Query*:

```
UPDATE copy_departments  
SET manager_id = 101  
WHERE department_id = 60;
```

Hasil:

```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
  
PS C:\Users\ASUS> sqlplus  
  
SQL>Plus: Release 11.2.0.2.0 Production on Tue May 10 08:06:56 2022  
  
Copyright (c) 1982, 2022, Oracle. All rights reserved.  
  
Enter user-name: system  
Enter password:  
Connected to:  
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production  
  
SQL> UPDATE copy_departments  
2 SET manager_id = 101;  
3 WHERE department_id = 60;  
  
1 row updated.  
  
SQL> |
```

Gambar 12.1 Mengubah *manager_id* pada tabel *copy_departments*

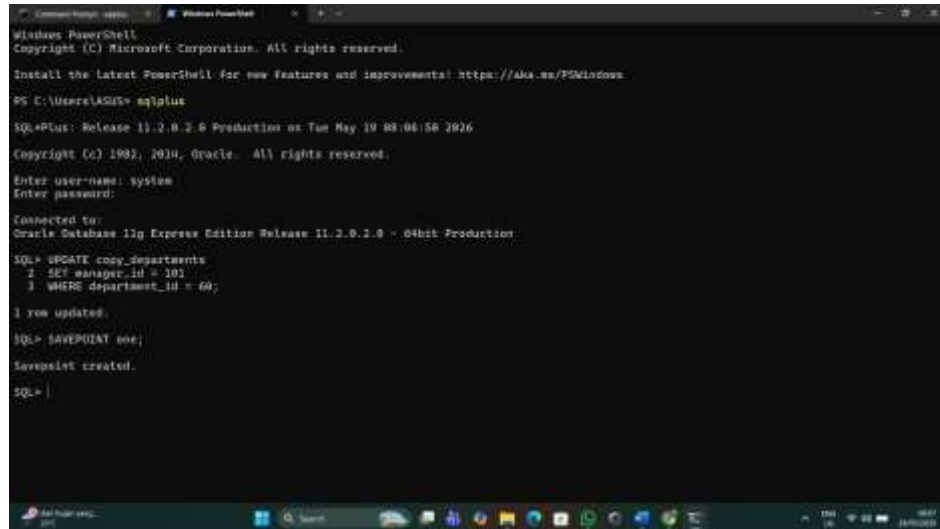
Penjelasan:

Eksekusi instruksi `UPDATE copy_departments SET manager_id = 101 WHERE department_id = 60;` berfungsi untuk melakukan modifikasi nilai pada atribut tabel *copy_departments*. Melalui kueri DML ini, kolom *manager_id* diperbarui menjadi “101” dengan pembatasan ketat hanya untuk baris yang memiliki nilai *department_id* bernilai “60”.

2. Query:

```
SAVEPOINT one;
```

Hasil:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ASIS> sqlplus

SQL*Plus: Release 11.2.0.2.0 Production on Tue May 19 08:06:56 2026
Copyright (c) 1992, 2014, Oracle. All rights reserved.

Enter user-name: system
Enter password:

Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

SQL> UPDATE copy_departments
1 SET manager_id = 102
2 WHERE department_id = 60;

1 row updated.

SQL> SAVEPOINT one;

Savepoint created.

SQL>
```

Gambar 12.2 Membuat SAVEPOINT one

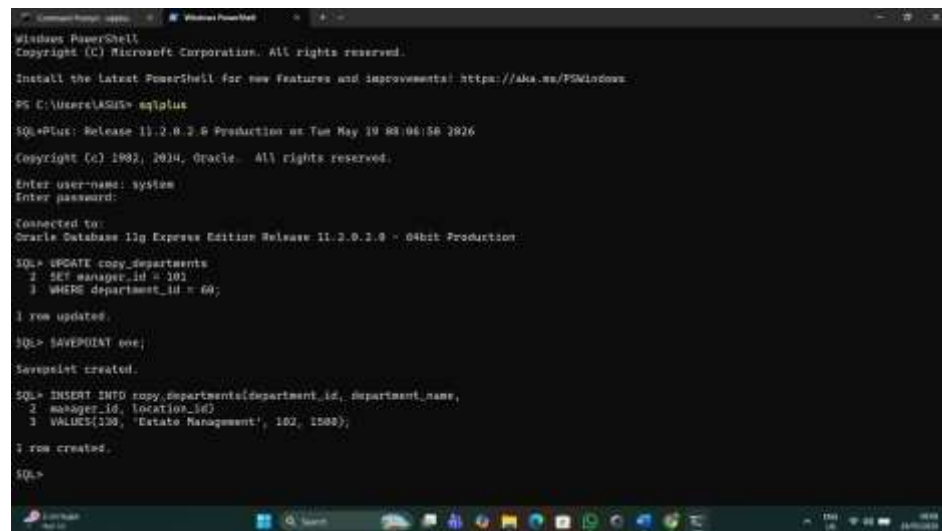
Penjelasan:

Sintaks `SAVEPOINT one;` dioperasikan untuk menginisialisasi sebuah titik penanda memori atau simpanan logis sementara di tengah berjalannya siklus transaksi. Penanda transaksi ini diidentifikasi dengan nama “one”, sehingga bertindak sebagai batas pemulihan apabila terjadi kesalahan eksekusi pada kueri-kueri setelahnya. Melalui penempatan *savepoint* ini, pengembang memperoleh fleksibilitas untuk membatalkan modifikasi data secara parsial hingga batas tahapan tertentu tanpa perlu mengorbankan atau mengulang kembali seluruh rangkaian transaksi dari awal.

3. Query:

```
INSERT INTO copy_departments(department_id,  
department_name,  
manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:



```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
  
PS C:\Users\AGUS> sqlplus  
  
SQL*Plus: Release 11.2.0.2.0 Production on Tue May 19 08:06:56 2026  
Copyright (c) 1992, 2014, Oracle. All rights reserved.  
  
Enter user-name: system  
Enter password:  
Connected to:  
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production  
  
SQL> UPDATE copy_departments  
2 SET manager_id = 101  
3 WHERE department_id = 60;  
  
1 row updated.  
  
SQL> SAVEPOINT one;  
  
Savepoint created.  
  
SQL> INSERT INTO copy_departments(department_id, department_name,  
2 manager_id, location_id)  
3 VALUES(130, 'Estate Management', 102, 1500);  
  
1 row created.  
  
SQL>
```

Gambar 12.3 Menambahkan data departemen baru

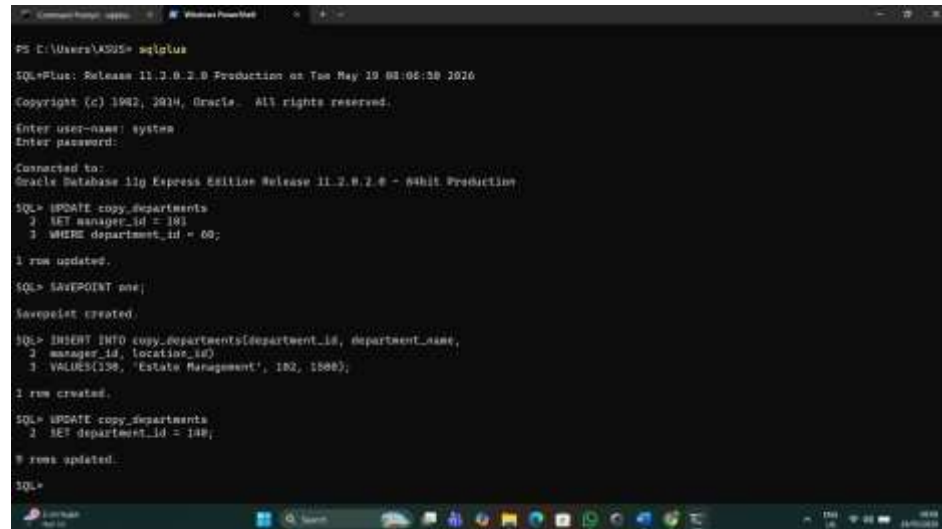
Penjelasan:

Perintah `INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(130, 'Estate Management', 102, 1500);` dijalankan untuk meregistrasikan satu record baris informasi baru ke dalam tabel fisik `copy_departments`. Komponen data yang disisipkan tersebut memuat rincian berupa kode `department_id` bernilai “130”, deskripsi nama departemen “Estate Management”, kode `manager_id` “102”, serta kode `location_id` “1500”.

4. *Query:*

```
UPDATE copy_departments  
SET department_id = 140;
```

Hasil:



```
PS C:\Users\ASUS> sqlplus  
SQL*Plus: Release 11.2.0.2.0 Production on Tue May 19 08:06:38 2020  
Copyright (c) 1982, 2014, Oracle. All rights reserved.  
Enter user-name: system  
Enter password:  
Connected to:  
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production  
SQL> UPDATE copy_departments  
2 SET manager_id = 181  
3 WHERE department_id = 60;  
1 row updated.  
SQL> SAVEPOINT one;  
Savepoint created.  
SQL> INSERT INTO copy_departments(department_id, department_name,  
2 manager_id, location_id)  
3 VALUES(130, 'Estate Management', 182, 1808);  
1 row created.  
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
9 rows updated.  
SQL>
```

Gambar 12.4 Mengubah department_id pada tabel
copy_departments

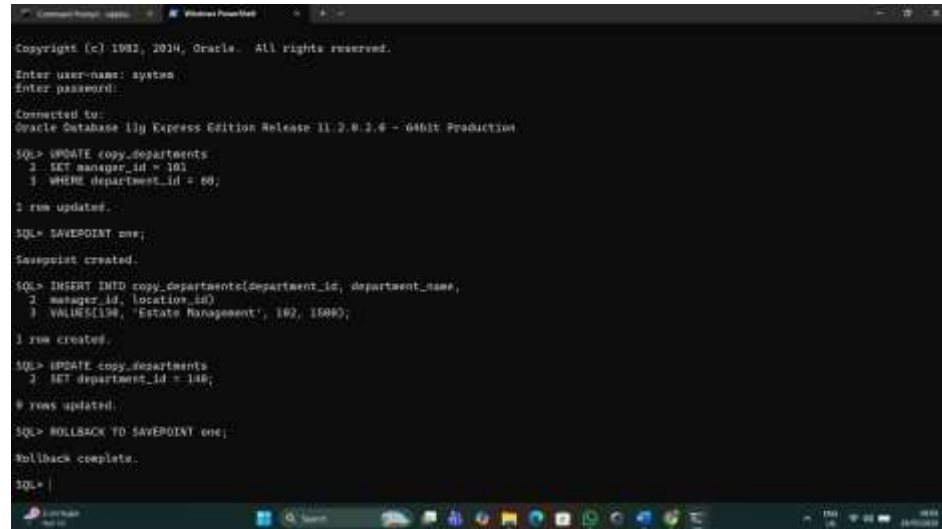
Penjelasan:

Kueri UPDATE copy_departments SET department_id = 140; dieksekusi dengan tujuan untuk mengubah nilai pada kolom department_id di dalam tabel copy_departments. Lantaran penulisan instruksi pengubahan ini sengaja tidak menyertakan klausa pembatas WHERE, sistem secara otomatis menerapkan perubahan secara menyeluruh, yang mengakibatkan seluruh baris data di dalam tabel tersebut berganti nilai menjadi “140”.

5. Query:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:



```
Copyright (c) 1982, 2014, Oracle. All rights reserved.
Enter user-name: system
Enter password:
Connected to:
Oracle Database 11g Express Edition Release 11.2.0.3.0 - 64bit Production

SQL> UPDATE copy_departments
  2 SET manager_id = 101
  3 WHERE department_id = 60;

1 row updated.

SQL> SAVEPOINT one;

Savepoint created.

SQL> INSERT INTO copy_departments(department_id, department_name,
  2 manager_id, location_id)
  3 VALUES(198, 'Estate Management', 102, 1500);

1 row created.

SQL> UPDATE copy_departments
  2 SET department_id = 148;

0 rows updated.

SQL> ROLLBACK TO SAVEPOINT one;

Rollback complete.

SQL>
```

Gambar 12.5 Mengembalikan transaksi ke SAVEPOINT one

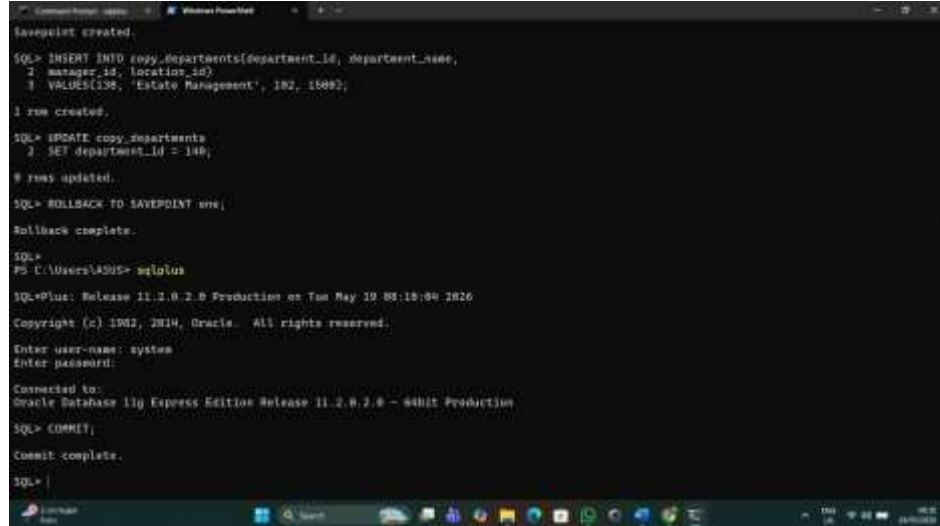
Penjelasan:

Instruksi `ROLLBACK TO SAVEPOINT one;` digunakan sebagai mekanisme pemulihan untuk mengeliminasi seluruh rangkaian modifikasi data yang terjadi pasca-pembuatan `SAVEPOINT one`. Karakteristik utama dari perintah ini adalah sifatnya yang selektif, di mana database tidak akan membatalkan unit kerja transaksi secara total, melainkan murni mengembalikan kondisi data ke status terakhir tepat saat penanda *savepoint* terkait diaktifkan.

6. Query:

```
COMMIT;
```

Hasil:



```
savepoint created.
SQL> INSERT INTO copy_departments(department_id, department_name,
  2 manager_id, location_id)
  3 VALUES(138, 'Estate Management', 182, 1608);
1 row created.
SQL> UPDATE copy_departments
  2 SET department_id = 148;
9 rows updated.
SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
SQL>
PS C:\Users\ASUS> sqlplus

SQL*Plus: Release 11.2.0.2.0 Production on Tue May 19 08:18:04 2026

Copyright (c) 1982, 2014, Oracle. All rights reserved.

Enter user-name: system
Enter password:

Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

SQL> COMMIT;
Commit complete.
SQL>
```

Gambar 12.6 Menyimpan transaksi dengan COMMIT

Penjelasan:

Perintah `COMMIT;` difungsikan sebagai instrumen validasi akhir untuk mengunci dan meresmikan seluruh perubahan data di dalam memori transaksional secara permanen ke dalam sistem penyimpanan fisik basis data. Pasca-eksekusi perintah komit ini, seluruh urutan perubahan yang telah dilakukan terkunci sepenuhnya dan tidak dapat dipulihkan atau dibatalkan lagi menggunakan instruksi `ROLLBACK`.

12.4 Latihan

Berikut adalah hasil dan pembahasan dari latihan yang telah dilakukan:

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO mahasiswa (nim, nama, jurusan, semester)
VALUES (2407112480, 'Maulana Malik Ibrahim',
'Teknik Elektro', '4');

SAVEPOINT satu;

SELECT nim, nama, jurusan, semester
FROM mahasiswa
WHERE nim = 2407112480;

SAVEPOINT dua;

DELETE FROM mahasiswa
WHERE nim = 2407112480;

ROLLBACK TO SAVEPOINT dua;

COMMIT;

SELECT nim, nama, jurusan, semester
FROM mahasiswa;
```

Hasil:

```

SQL> INSERT INTO mahasiswa (nm, nama, jurusan, semester)
2 VALUES ('SIMPULING', 'Maulana Ralla Simpul', 'Teknik Komputer', 'K1');
ERROR at line 2:
ORA-00917: invalid character

SQL>
SQL> SAVEPOINT data;
Statement created.

SQL>
SQL> SELECT nm, nama, jurusan, semester
2 FROM mahasiswa
3 WHERE nm = 'SIMPULING';
no rows selected

SQL>
SQL> SAVEPOINT dua;
Statement created.

SQL>
SQL> DELETE FROM mahasiswa
2 WHERE nm = 'SIMPULING';
0 rows deleted.

SQL>
SQL> ROLLBACK TO SAVEPOINT dua;
Rollback complete.

SQL>
SQL> COMMIT;
Commit complete.

SQL>
SQL> SELECT nm, nama, jurusan, semester
2 FROM mahasiswa;

      NM      NAMA      JURUSAN      SEMESTER
-----
1 Maula Maula      Teknik Komputer
SQL>

```

Gambar 12.7 Mengelola transaksi data pada tabel mahasiswa

Penjelasan:

Rangkaian manajemen kontrol transaksi diterapkan secara komprehensif pada tabel mahasiswa dengan mengintegrasikan kombinasi perintah operasional DML dan TCL yang meliputi INSERT, UPDATE, DELETE, SAVEPOINT, ROLLBACK TO SAVEPOINT, serta pengesahan akhir melalui COMMIT. Di dalam sesi kerja ini, instruksi INSERT berperan dalam menyisipkan record data baru, sedangkan perintah UPDATE difungsikan untuk memodifikasi nilai atribut spesifik pada tabel mahasiswa. Di samping itu, perintah DELETE dioperasikan untuk mengeliminasi baris record tertentu dari tabel. Demi menjaga alur kerja transaksional dari kesalahan logika, klausul SAVEPOINT dipasang sebagai penanda memori sementara sehingga pemulihan status data dapat diarahkan ke titik tertentu secara aman menggunakan perintah ROLLBACK TO SAVEPOINT. Seluruh rangkaian aktivitas data yang dinilai telah valid kemudian dikunci secara absolut ke dalam sistem database menggunakan perintah COMMIT pada akhir sesi.

12.5 Kesimpulan

Berdasarkan praktikum, dapat disimpulkan bahwa *Transaction Control Language* (TCL) berfungsi untuk meregulasi unit kerja transaksi di dalam basis data. Melalui instruksi `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`, pengguna memiliki kendali penuh untuk mengawasi status modifikasi data sebelum diterapkan secara permanen ke dalam penyimpanan fisik. Mekanisme transaksi ini memastikan bahwa eksekusi kueri manipulasi seperti `INSERT`, `UPDATE`, dan `DELETE` dapat dikelola dengan jauh lebih aman, terarah, dan terisolasi dari risiko kerusakan data akibat kegagalan sistem di tengah proses.

Secara teknis, perintah `SAVEPOINT` sangat membantu dalam menciptakan titik simpan logis sementara, sehingga pembatalan data dapat dilakukan secara parsial melalui perintah `ROLLBACK TO SAVEPOINT` tanpa mengorbankan kueri lain yang sudah benar. Di sisi lain, `ROLLBACK` digunakan untuk membatalkan seluruh perubahan yang belum disimpan, sedangkan `COMMIT` bertindak sebagai validasi absolut untuk mengunci data secara permanen. Didukung oleh prinsip *read consistency* dan fitur *locks* untuk mencegah konflik multi-pengguna, penguasaan TCL menjadi pilar fundamental dalam menjaga keamanan, ketepatan, dan konsistensi informasi di dalam *database*.